NESS, the Neural-Symbolic Sandwich

A new direction for making transformers AGI-friendly — via judiciously mixing contir symbolic program learning into open-weights transformers.



BEN GOERTZEL
SEP 29, 2026

12 4 2

I'm going to write here about some technical work I've been playing with over th few weeks – which is going quite well and which, if it keeps going well, might tu to have some rather interesting consequences. It's an AI architecture I've been c NESS — the **NEural-Symbolic Sandwich** — and basically, it's a way of taking the open-weights transformers we already have and restructuring and retraining *par* them in a way designed to make the whole thing more AGI-ish and a better parti in broader AGI architectures.

I'll go into moderate depth here, and link to paper drafts going deeper at the end let me start with the TL;DR for technical readers: ***NESS takes a pretrained transfor freezes most of it, distills its top layers into a small adaptable bridge, and puts three mor layers on top: a symbolic layer of executable programs and statistical "readers" that wor structured records and history, a probabilistic combiner that learns how much to trust ea them and how their evidence should interact, and a small neural cap that turns the coml evidence into the final prediction.***

So – the bottom layer changes slowly or not at all, it's just derived from the vanill transformer base. The four layers above it keep learning ongoingly and continual the time – and the symbolic one in the middle is built to plug into a diversity of c AI tools, including Hyperon's AtomSpace, MeTTa, PLN and program evolution.

What I've run so far are just tiny prototypes — around half a million neural para — but the results have been interesting and the math suggests the methods will scale up. At this tiny scale,

- The discovered symbolic programs improve prediction by about 7–8% over matched purely-neural controls
- *There are meaningful long-term memory properties*: When the system is trained text, then only on Python, then text again, the neural bridge forgets a good d but the whole system reverses roughly 88–94% of the prose loss it had incurr largely because the symbolic layer held its ground.

None of that is anywhere near AGI yet of course; and even with promising math, scaling algorithms and architectures always needs some trial and error tuning, a there can be hidden rocks.

However, my intuition is fairly fired up here – this sandwich feels like the shape transformer that can keep learning ongoingly, and that speaks the same symboli language as a logical reasoning engine and the rest of the Hyperon architecture. Plugging a bigger NESS network into an Omega agent along with Hyperon Atomspaces and associated symbolic and evolutionary algorithms, one would ob an even more powerful thing than the current Omega system: An agentic loop in which the neural and symbolic dynamics involved in each pass through the cont group are actively cooperating and communicating, sharing internal representati rather than just feeding each other outputs as inputs.

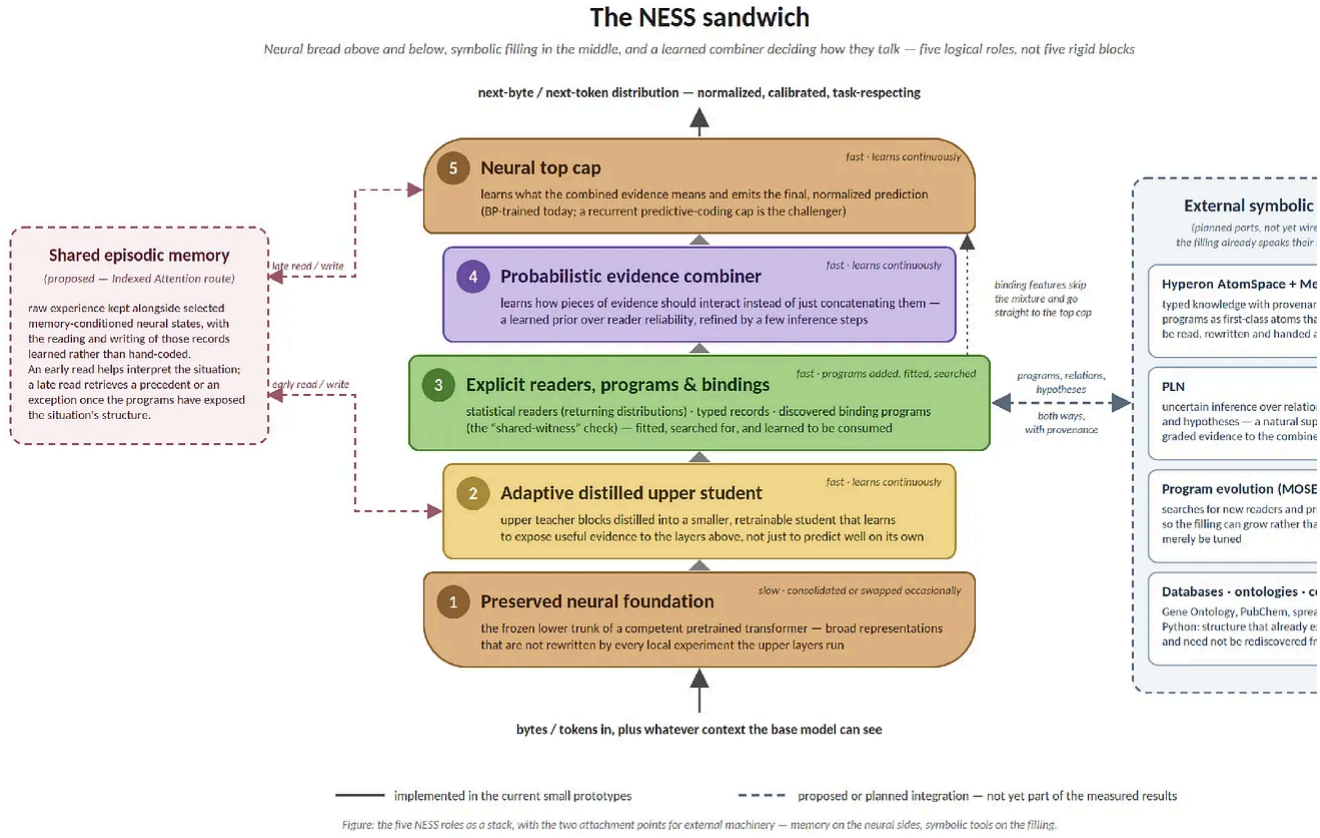


Figure 1. *The NESS sandwich: neural bread above and below, symbolic filling in the m and a probabilistic combiner learning how the two should talk to each other. The filling the natural port through which external symbolic machinery — Hyperon's AtomSpace MeTTa, PLN, program evolution, and plain databases, ontologies and code — can ent predictor, while a shared episodic memory would attach to the two neural sides. Sol connections are implemented in the current prototypes; dashed ones are proposed or pl*

Where transformers sit in my picture AGI

As anyone who follows my work will know, I don't think LLMs are the holy path AGI. I do think they can be very useful components of AGI systems — which is v Omega, the agent framework my colleagues and I have been building, we have ar agentic loop wrapping LLM calls, and along the way a bunch of calls into Hyperc symbolic working memory and long-term memory (and, more experimentally, medium-term memory), with symbolic reasoning going on inside each go-round

loop. We're gradually tweaking Omega to shift more and more of the goal pursui self-understanding into the symbolic side and to rely less and less on the LLM. Ultimately, in a more mature Omega network, I see the LLM as a communicatio for going back and forth between structured knowledge and English or other hu languages, and as a sort of lossy, questionable, but nonetheless valuable knowled oracle. Hyperon, with its symbolic reasoning and evolutionary learning and so fo should do the rest.

But since there is an LLM in the loop, even if it isn't the crux of the Omega loop' intelligence, I'd like it to be as AGI-ish as possible. And there are a number of th stopping current LLMs from being fully human-level-AGI-ish. The basic representation of knowledge they have is overly tied to the specific information they've been fed, so they do a bit too much recombining of details from the train data and a bit too little wild, general-purpose abstraction and creative leaping be those details. That's a deep issue and I'll come back to it. The other major shortc — the one NESS is most directly aimed at — is continual learning.

The frozen-model problem

The way LLMs work today, you can't do weight learning and activation-space lea and inference all in the same process at the same time. You train the model, you the weights, and then you have a frozen model — Opus 5.6, Astra, whatever — th do inference with. You can get some learning out of it in the activation space, via context learning, which is super cool and amazing as far as it goes. Then, to mai context beyond what the activation space will hold, the model gets propped up w bunch of external memory devices, some of which work well and some of which badly. Anyone who uses these systems regularly can see that the props aren't gre

Part of why Omega works as nicely as it does is that Hyperon's symbolic memory serves as a much better long-term memory prop for the LLM than the usual bag retrieved snippets — better than a prop, really, since we're doing interesting reas over what's stored rather than just looking things up. To be fair, not every extern memory used in the standard LLM world is an improvised bolt-on: retrieval-

augmented generation can train the retriever and the generator jointly, and the r
wave of work on "test-time" neural memory is already exploring alternatives to fi
deployment.[3] But in all of these it's still the LLM's shortcomings being compens
for from outside. The issue is how tightly remembering, interpreting, reasoning a
learning are woven together — whether they're one activity or four loosely coupl
ones.

For the reader less intimate with modern neural voodoo, let me be precise about
the shortcoming lives. Backpropagation is the method used to adjust a neural
network's weights based on error signals propagated backward from its output.
Transformers are an architecture, a particular way of wiring a network together
around attention. Backpropagation is a powerful algorithm, dating back to the 19
but it has some particular shortcomings too.

For many complex neural architectures it just won't do the learning in any tracta
period of time – it "won't converge" – and this has guided the field away from so
neural architectures and toward others. In particular it has directed the field tow
architectures with relatively few and simplistic recurrent connections, because t
much recurrence tends to make backprop choke.

Backprop-based neural networks are also commonly prone to "catastrophic forge
– after you train them a while they start to exceed their memory capacity and can
suddenly forget a lot of their knowledge very rapidly. They don't gradually lose tr
old information to make room for new information. This problem is not unique t
backprop – it exists for example in Hopfield associative memory neural networks
well, though there it can be worked around with tricks like reverse learning. This
problem is the root of the current divide between training and inference in LLM
is the reason we freeze a model after its weights are trained, and then after that d
let the weights change, but instead allow only the "in context learning" that can
happen in the activation space

A trivial example of the stupidity this division leads to: at the moment, all the LL
I'm using inside my hive of AI agents were evidently trained before Telegram
introduced its bot-to-bot API. So on the first pass, these agents think it isn't pos

for Telegram bots to talk to each other — even though they are, at that very mom
controlling Telegram bots that are talking to each other. Usually they figure out
they're doing and then look on the Web to get the new information their models
incorporate. But every so often one of them consults its underlying model, the m
says no, Telegram has no bot-to-bot API, and the agent loses track of the fact tha
currently using Telegram's bot-to-bot API. The agents can store the true fact in t
AtomSpace, or in a vector memory; what they can't do is update the LLM, so the
keeps offering the same wrong information over and over, and now and then they
believe it.

That's a trivial case, since it's just an empirical external fact, and storing it in a
database mostly fixes it. The thing is, there are loads and loads of subtler cases li
that aren't so easy to iron out with a database entry — cases where what needs to
change is how the model interprets a whole class of situations, not one fact it can
up.

The kind of memory I find myself most concerned with is what I'd call *medium-te*
memory: the evolving knowledge of a project, a relationship, an investigation, an
unfinished argument. It lives in the awkward middle ground between what fits in
current context window and what's been consolidated into a general skill. A bag
retrieved snippets doesn't maintain the distinction between what happened, wha
thought happened, and what subsequent evidence changed my mind about — an
maintaining that distinction is a big part of what medium-term memory is *for*. Fo
robust self-modeling, other-modeling and world-modeling, I want that continuit
into the learning process itself: an agent ought to be able to track which of its ow
methods have failed, which of its commitments are still live, what another agent
to know, and why its own confidence in something went up or down. (For now yo
just consider this a functional claim, not a claim about machine consciousness; I
leave those questions for other posts.)

What I would ideally like inside an Omega loop, then, is just a transformer that k
updating its weights as it goes along. The human brain does this. We have long-t
potentiation and other such mechanisms; the weights on our neural connections

updated in semi-real time as activation spreads around them. The updating tend slower than the activation spreading, but it's all wrapped up in one process rathe being a separate offline phase.

Predictive coding, and the long road t making it work at scale

I'm fairly confident one can train a multi-trillion-parameter transformer, with sc variation on the current architectures, in a way that supports continual learning, there are probably many ways to do it. The one my team at SingularityNET and E Labs has been experimenting with most is predictive coding, which I've talked al before but will recap here very briefly for newer readers.

Yes, believe it or not, you can train neural nets using algorithms other than stanc backprop. In predictive coding — PC — instead of sweeping a weight-adjustmer through the whole network from top to bottom, each neuron does its own learni trying to predict what the neurons in nearby layers are doing. There's then a sett in process: neurons here adapt to neurons there, neurons there adapt to neurons over and over, and through this sometimes rather chaotic iterative process the ne as a whole settles into a functional attractor. Only once things have settled do th persistent weights get updated, via a local rule rather than a global gradient. Wh train networks this way you often find they don't suffer nearly as badly from catastrophic forgetting.

That said, most of the PC literature has been on vision rather than language, and in practice is still catching up to backprop. In our SingularityNET research team we've gotten a predictive-coding transformer to work OK at the GPT-2 scale. Generally what we find in our early experiments is that a PC network's continual learning performance is better than a comparable backprop network's, but not as as we'd like. But there is so much plain room for improvement!

I think of the situation with PC today as similar to the long road backprop itself traveled. When I was teaching backprop neural nets in the '90s, it took somethin

three hours to get a 35-neuron recurrent backprop network to converge to anythi
Part of that was the Solaris workstation I was using, which was primitive compa
a modern smartwatch, but part of it was that recurrent backprop on a richly con
architecture was just the wrong thing to be doing, and it took years to figure out
tricks — filtering between layers, limited doses of recurrence, attention mechani
and all the rest. Those tricks won't necessarily be the same tricks for PC. So we'r
trying to rush through that process for predictive coding, and I do think that onc
worked through we'll end up with something with much more brain-like propert
including better continual learning and the ability to train locally, neuron by neu
across a decentralized network of machines. Our FabricPC toolkit is speeding th
along considerably, and as far as I can tell we're the first group seriously trying t
make PC work at large scale in practice, as opposed to demonstrating interestin
phenomena at small scale in academia.

But — all that said — I've gotten impatient. That work is coming along, and it's
fascinating, and a huge breakthrough could well come any day … but I'm in a hur
make sure the open and decentralized and moral-agency-oriented approach wins
AGI race, so I don't want to depend too desperately on any particular thread of
research. So I've started to think about alternate, complementary paths as well…
is where NESS came from.

Keep the expensive part, replace the top

So what I've been musing in the background for months is: ***OK, take a big open-w
transformer — of the kind the Chinese labs keep releasing with such admirable generosi
freeze most of it, leave it as it is, and replace the top of it with something more flexible, r
brain-like, more neural-symbolically linkable, more capable of continual learning.***

Maybe the base is basically a model of what's on the web, and maybe it's fine to r
that every three or six months, since the web doesn't change all that fast. Maybe

even fine to leave it frozen even longer if the top bits of the network are built the we really want.

There's an obvious economic argument for this sort of approach. The useful pret models already embody an immense amount of computation and engineering — billions of dollars' worth at the frontier — and we don't need to repeat that inves before investigating a better organization of adaptation.

There are roughly similar precedents of course: parameter-efficient methods suc LoRA keep a large model essentially fixed while training a small set of extra parameters, and knowledge distillation trains a smaller "student" network to reproduce the behavior of a larger "teacher."[4]

The device NESS uses is what we call a **cap** — a learned module attached on top model that already works, which can read the base's internal representations and predictions, consult additional evidence, and improve the output. I should say up that updating fewer parameters narrows the learning problem without making th base's forward pass, or retrieval, or recurrent inference free. "Efficient" here shou not be read as "cheap."

And I suspect the slow-base, fast-top split may be more than a transitional compromise. Perhaps broad perceptual and linguistic competence is the kind of that *should* change slowly, while the machinery that interprets new experience, constructs abstractions and coordinates action should change fast. A stable base the fast-changing parts something reliable to build on. Upgrading it, when that happens, means checking that old memories and learned interfaces still mean wh think they mean — the same problem any long-lived piece of software faces whe dependency changes underneath it. The interesting question is whether the who system keeps learning coherently while its parts learn at different speeds.

The first thing I tried was simply training a predictive-coding cap on top of a sta transformer. That worked OK at small scale, but not as well as I wanted. Next I t distilling the base network — taking a backprop-trained transformer and, using homotopic distillation procedure, converting it into what's called an error-PC

network, a kind of predictive-coding network that looks a lot like a backprop net — and then putting a more flexible PC cap on top of that, co-training the distilla with the learning of the cap. That worked a little better. Still not as well as I wan And that is what led me to the approach I'm playing with now.

NESS, the Dagwood Bumstead Sandw of AGI

I call it the neural-symbolic sandwich — NESS, pronounced like the Loch Ness monster. The LLMs I was using to help develop it don't like the sandwich metap You're not just piling things on top of each other like a sandwich, they told me; i about the interfaces between the layers. And of course they're right, but LLMs ha bad sense of poetry. If you're old enough to remember the newspaper comic Blor Dagwood Bumstead used to eat these towering multi-layer sandwiches, and that what I'm envisioning: five layers, each doing something different, with the intere stuff happening where they meet.

The five layers, bottom to top, are: a frozen pretrained transformer; a distilled to couple-of-layers of that transformer, adapted into a bridging layer; a symbolic lay which is the most interesting part; a probabilistic combiner; and a neural cap tha produces the output distribution. The bottom layer is fixed — for the small cases been running I train it myself, but for a big case you'd take an existing open-weig model — and the other four are all co-trained for the purpose. What's the loss function? You can play with that, but for now it's standard transformer stuff, bec we're not trying to train a full AGI here. We're trying to train a transformer that decentralized-trained and do robust continual learning, to plunge into an Omega alongside the other AI components I have. If it turns out to be an AGI on its own won't complain, but that's not what I'm looking for. Figure 1, up at the top of this shows the whole stack; the reader who made it this far is rewarded with a brief explanation of what each layer does.

Layer 1: the frozen foundation

Keep the lower part of a competent pretrained model. It supplies broad representations of language — or images, or whatever the base was trained on — without being rewritten by every local experiment the upper layers run. Freezing removes one big source of interference between old and new learning. It doesn't guarantee the complete predictor can never forget, since the layers above can sti drift; as we'll see, they do.

Layer 2: the distilled bridge

The representation most useful for the original language-modeling objective isn necessarily the most useful interface to a symbolic system. So rather than keepin whole base frozen, we take the top couple of transformer blocks and distill them smaller, adaptable student, trained together with the more unusual things that si above it. You can do the distillation with backprop, or with error-PC, or whateve student's job isn't just to predict well on its own but to expose useful evidence to rest of the sandwich. In the small reference implementation two teacher blocks became one narrower student block; how much to distill, how far to compress an where to cut are experimental variables rather than commandments.[1]

Layer 3: the symbolic filling

The middle layer is a diverse ensemble of symbolic programs, and I've experimer with a whole bunch of options for what goes in there. The programs take neural as input — the output of the distilled bridge — and they also take history as inpu an important point is that they have to use context to produce their outputs. The form their own context from a symbolic history, and they can also look at the cor fished up by the frozen base's attention mechanism and use that to grab the relev bits of symbolic history. Neural and symbolic attention, both aimed at a symboli record of what's happened. Over all of this you learn an ensemble of programs.

Why put symbols in the middle at all? A symbolic system has a straightforward representational advantage: you can add a fact, revise a relation or compose a ne program without retraining a big network to encode that particular addition, so

incremental knowledge and flexible long-term memory are ordinary operations r than heroic ones. And these operations are close to the work we want AI to do. C has variables, scope and data flow; mathematics has quantified statements and p dependencies; spreadsheets and databases have fields, identities, formulas and jc resources like the Gene Ontology and PubChem hold biological and chemical knowledge in structured form.[5] Why translate all that into prose and then ask a model to rediscover, from the prose, relationships that were explicit in the sourc Better to let the language model help interpret the question, let an executable pr follow the actual dependency, and let a learned consumer decide what the result in context.

One particularly interesting way to build these programs is with semantic frame text is coming into the transformer. Run it through an entity extractor and a sem relation extractor, so you get predicate-argument structure: if the sentence is "Be walked through the field," the transformer is doing its neural thing, while the rel extractor says Ben is an entity, the field is an entity, and there's a walked-throug relationship between them. Modern transformers extract this kind of thing very robustly — we could do it even before transformers, with the Stanford and Berke parsers of pre-transformer computational linguistics — and in principle the extr could be co-trained with the rest of the network, though for now existing tooling works fine.

Given the relations, the symbolic programs become combinations of semantic relations with properly aligned variable bindings, which is to say they represent frames and events. For instance: one actor sends a message, another actor receiv there's a channel it's sent on, there's an encoding scheme, and the sender and the receiver are both using the same encoding scheme. That's a bundle of semantic relations involving a couple of actors, held together by shared variable bindings, "sending a message between a sender and a receiver over a channel using a mutu understood encoding" is a little program that lives in the symbolic layer and can recognized as something coming through the input. Even in the small examples run there are hundreds or thousands of these; a frontier-scale model would have insane number of them.

A toy version of the same idea, at the scale the current prototypes run at: Dan ne object that's in a locked cabinet; someone Dan trusts is willing to help; someone key that fits the cabinet. Those facts alone don't get Dan his object. What's need that ***the trusted helper be the very person holding the key that opens the cabinet contain object*** — one individual satisfying all three roles at once. A small program enforc that repeated identity exactly, where a neural predictor can only approximate it, NESS's prototype program search found exactly this sort of "shared witness" computation useful for prediction.[1,2]

Since the whole point of putting programs in the middle rather than just more n is that you can read them, let me show what the search in the latest prototype ca with. The toy world it lives in has 20 entities and 9 relation types — trusts, helps holds, opens, inside and so on — and the program grammar builds candidates ou typed paths through those relations, plus "shared-witness" joins that insist two p land on the same entity. Three of the programs the search selected form a rather pleasing ladder (conceptually, I mean; I'm not claiming the search built them in t order).

The simplest, program 0 — call it *direct access* — is holds(A, K), opens(K, C), insic C): the actor A holds a key K, that key opens a container C, and the object O beir sought is inside that container. Three relations, with the same K and the same C wherever they appear.

Program 398, the *helpful holder*, is helps(H, A), holds(H, K), opens(K, C), inside(O, now it isn't the actor holding the key but some helper H who is helping the actor and it has to be the same H doing the helping and the holding.

Program 681, the *trusted helper*, is trusts(A, H), helps(H, A), holds(H, K), opens(K, inside(O, C): the helpful holder with one more relation bolted on, namely that the helper also be someone the actor trusts. Five relations, one helper, one key, one container, all tied together by the repeated variables. Each program returns 1 if t some way to bind its variables to actual entities so that every one of its relations at once, and 0 otherwise. The repeated variables are the entire point: it isn't enou that someone helps Dan and someone holds a suitable key, it has to be the same

someone. Program 681 was retained in all three full search runs, which is about a clear a vote as a search procedure can cast.

Here is how it plays out on an actual held-out world from the archived test set — names and relations, with only the relevant edges shown. Dan seeks the object G Dan trusts Ivo; Ivo helps Dan; Ivo holds the key Pia; Pia opens the container Joy; is inside Joy. Executing program 681 amounts to a set intersection: the people Da trusts, {Ivo}; the people who help Dan, {Ivo}; the people holding a key to wherever is, {Kai, Ivo}. The intersection is {Ivo}, so the program returns 1 — one identical h satisfies the trust, the help, and the whole key–container–object chain.

Now swap two trust endpoints and leave everything else alone. In the original wo Dan trusts Ivo and Eli trusts Eli; in the swapped world Dan trusts Eli and Eli trus Ivo. The local text is the same, the word counts are the same, every entity has the number of incoming and outgoing relations, and the cropped prefix the neural la see — "… Qin. Dan seeks Gus. Routine. Then " — is byte-for-byte identical. Prog 398 still returns 1 in both worlds, because Ivo still helps Dan and still holds the k Program 681 returns 1 in the original world and 0 in the swapped one, because th of people Dan trusts is now {Eli}, and Eli is not the one with the key.

And the prediction follows the binding. The next event in this world is Dan askir aid, so the next byte to be predicted is the "A" of "Asks aid." Averaged over three models, a context-only version of the system — one that sees the same text but g program evidence — puts 52.3% on that byte in the original world and 43.9% in th swapped world; it barely notices that anything changed. The system with the full discovered program bank goes from 85.4% to 18.0%. For reference, the process th generated the world, which the models never see as a training label, assigns 88.4% 17.2%: the program-equipped system is tracking the real structure of the situatio while the context-only one is guessing from surface statistics. This is one illustra pair, selected using observations only, and not an aggregate result — the aggrega come below — but it shows in miniature what the programs are doing: an exact, inspectable, shared-identity computation, moving the next-byte forecast in a way neural predictor on its own could only approximate.[2]

How do these programs get found? Not by backprop. The search learns program syntax by search and program consequences by learning. It enumerates candidat from the typed grammar (1,209 of them in the latest run), screens them by how w their features line up with what the neural predictor is currently getting wrong, f small context-specific consumer for each of a dozen nominees per round, and ad the one or two per seed that improve held-out prediction after a charge for comp Then it recomputes the residual and searches again. The structure of a program discrete and you can print it out; what its output means for the next byte is some the layers above learn.

In the current implementation, the simplest programs are what we call *readers*. S readers return complete predictive distributions learned from statistics — a tabl which bytes tended to follow a particular kind of context. Other programs return features rather than distributions: whether a shared-binding condition holds, wh entity satisfies a relation, what a query against a store came back with. So learnir this layer is more than adjusting weights: we can fit a reader's table, search for a reader or program, and learn how the layers above should consume the output. C caveat: the binding experiments so far work from supplied typed records, structu facts handed to the system, and don't yet demonstrate automatic extraction of arbitrary symbolic knowledge from raw prose. Bolting the frame-extraction pipe described above onto the prototypes is one of the obvious next steps.[1,2]

The statistical readers have the same see-through quality. One reader the early s generated refined a plain "current word prefix" reader by keying on the previous as well, so that the prefix "c" after "the" and the prefix "c" after "I" got separate byte tables — about 12,000 stored keys in place of the parent's 11,306 — with spa entries backing off to the parent's table. Of the 98 candidate readers that search that was one of the two it kept. It is a new conditional distribution the system lea to consult, and you can open the table and look at it, which is more than one can for a weight matrix.

The symbols here aren't a rationalization generated after the answer to make it l reasonable; they participate in producing the answer. And because they're progra

rather than activations, a discovered program can be inspected, reused, tested ur changed bindings, and eventually handed to other components — which is wher Hyperon comes in, and I'll get to that.

Layer 4: the probabilistic combiner

The fourth layer maps the outputs of the symbolic programs into the inputs the t neural cap wants. When I looked at this properly I realized it's a probabilistic programming problem, which is cool: you're trying to transform the distribution outputs of these programs into the distribution of inputs the top layer needs in o to give the desired output. A pile of correct computations is not yet a calibrated prediction. Several programs may be reusing the same evidence, so naively addin their votes double-counts it; a rule may apply under only one reading of the inpu statistical reader may be reporting from a context it has barely seen. The system learn how the evidence should interact, rather than concatenating everything an hoping the top layer sorts it out.

For now I train a simple probabilistic combiner, and you can do that with a varie mixture models: choose among a small number of evidence-use strategies and su their forecasts according to learned probabilities, with no training label ever hav say which strategy was the "correct" one. The latest prototype is a bit more speci than that — a learned prior over how reliable each reader is, adjusted through a f steps of inference using outcomes the system has already observed. Down the ro layer can be an arbitrarily complex learned probabilistic program; what we have a tractable instance of the bridge we need.[1]

Layer 5: the neural cap

The top layer is a neural net again, with a few layers of its own, giving the output probability distribution that ultimately gets judged and whose loss gets calculate learns what the combined evidence means for the task, translating a crisp logical result into a graded adjustment and handling interactions the explicit programs express. Ultimately I want this to be a predictive-coding net, and I've experimen

with PC, backprop and other functions up there. A backprop-trained cap is a per legitimate NESS model and an essential comparison — we'd be fooling ourselves only ever ran the PC version. Whatever learns the cap, its output has to respect t task: for language modeling the final token probabilities must form a properly normalized distribution.

I should stress that these are five *logical roles*, not a rigid pipeline. Neural context available alongside symbolic evidence at every stage. In the current implementat the distribution-valued readers enter through the mixture in layer 4, while the discovered binding features go straight to the final neural correction in layer 5. T sandwich describes how the interfaces cooperate; it doesn't demand that informa be thrown away at every boundary. (So the LLMs had a point. It is about the inter It's still a sandwich.)[1]

What the small experiments show, an what they don't

I've been training this on very small networks, and it works interestingly well. Th work grew out of a sequence of small experiments run under the internal label H which tested a number of different mechanisms and shouldn't be read as one ste climbing benchmark curve. Some earlier lexical experiments found that readers generated by search could beat fixed, hand-specified readers — including one refinement that conditioned on both the current word prefix and the preceding w Other runs found that more search, or a more elaborate consumer, didn't automa help, which is the kind of negative result it's good to have in hand before scaling anything up.[2]

The most recent "binding reference" model is deliberately tiny: about **454,000 deployed neural parameters**, with the external reader tables and other storage co separately. It sees a local context of 35 bytes, and evaluation scores every byte of byte continuation. These are controlled mechanism tests, not frontier results, an ask you to hold that frame through everything that follows.

The most clear-cut result concerns whether discovered programs help predictior the controlled co-training experiment we ran three inherited seeds with eight branches per seed; corresponding branches began from identical predictions anc received matched additional training examples and matched numbers of updates that the one systematic difference between a branch and its control was the pres of the program evidence. Against their respective no-program controls, the disco binding programs reduced the loss on complete structured continuations by **8.2(with a frozen core and 6.93% with a jointly trained core**. The improvement depe on the bindings being right — damaging or swapping the program evidence mac predictions worse, not merely no better — and the neural controls saw the same underlying observations, so the programs weren't smuggling in extra facts.[2]

I read this as evidence that an explicit computation can earn its place inside a ne predictor, on the predictor's own terms. It is not evidence that symbolic features supply facts the network couldn't otherwise get at. And there's a result here I'd h preferred to go the other way: co-training the core alongside the programs did n increase the programs' incremental benefit. That interaction came out slightly negative, even as performance on ordinary prose improved. So we have useful cooperation to build on, but not yet the stronger synergy — the kind where the n and symbolic sides make each other better — that the whole approach is ultimat after.

Text to Python and back again

The little experiment that got me most excited is the one that speaks directly to continual learning. I trained the network on text, then trained it more on Python then trained it on text again, and then checked: did it forget everything? Concret **200 updates on text, then 800 updates on code only, then 200 updates on text**. N examples were replayed during the code phase, and no checkpoint was restored v text returned; the system just had to cope. The lower trunk and the reader tables stayed fixed throughout while the upper layers adapted. We compared an *anchore* condition, which penalized the upper system for straying too far from the pre-co predictor on current-domain inputs, against an otherwise matched *unanchored* or

twelve trajectories in all, with disjoint sets of standard-library modules for Pytho training and test, so the code results aren't memorization of specific files.[2]

Here's the prose result for the program-equipped models, averaged over three se The unit is bits per byte, a standard measure of how well a model predicts text: lo means the predictor assigned higher probability to the bytes that occurred.

Prose test, by training stage

	With anchoring	Without anchoring
Before code training	2.4669	2.4669
After code-only training	2.8012	2.9458
After returning to text	**2.4872**	**2.5237**

Held-out prose loss in bits per byte; three-seed means, all 29 continuation bytes scored. S NESS empirical record, text–code–text experiment.[2]

Meanwhile, held-out code loss fell by about **32% with anchoring and 33% witho** — so retention wasn't being purchased by refusing to learn the new domain. Afte training resumed, the complete system reversed roughly **94% and 88% of the pro deterioration**, respectively.[2]

That's encouraging resilience, though it doesn't yet show all the robustness I wo hope for. There is still plenty to play with, and plenty of work to be done. **Recove not the same thing as uninterrupted retention.** Prose got appreciably worse dur the code phase; neither mean fully returned to its starting point; and returning t kept only about 42% and 38% of the newly acquired code gain. There was interfe in both directions, which is what one expects from any system that learns at all.

What's most interesting, though, is what happened inside. Layer 2, the distilled p the base transformer, forgot a lot: when the system learned Python it lost a good chunk of its text ability, and in the unanchored model, after text came back, the u transformer component taken on its own was still about 0.840 bits per byte wors prose than before the code phase. But the complete predictor was only about 0.0 worse. The symbolic layer, of course, doesn't forget in the same way — you don't to delete the programs and reader tables for text just because you're training on Python — and because it held its ground, the whole network got its integrated prediction ability back fairly quickly once text came back. The preserved readers the learned combination are the plausible compensators, though this test didn't i their causal shares.[2]

That the continuity symbolic systems get for free carried over to the integrated n symbolic system was not obvious. It could easily have gone the other way: you tra thing on text and then on Python and it just gets discombobulated and can't segu back. It segued back nicely. Two caveats, though. The binding programs in parti were inactive on ordinary prose and code, and the no-program branches showed similar prose retention, so those particular programs don't get the credit. And th upper-transformer diagnostic is an internal measurement, not a separately traine conventional-transformer baseline matched for size and data. What the result su is a promising division of labor among the layers. All this doesn't do is declare catastrophic forgetting solved – but I believe it's a highly promising direction, ar won't be surprised if we make further progress in this direction very rapidly.

Why richer programs may bring predictive coding into its own

Moving toward systems in the broad capability class of GPT-4 or GPT-5 changes kinds of tasks worth attempting, and not merely by enlarging the vocabulary of s rules. Useful programs at that level may need nested quantification, multiple sco uncertain reference, and compositions of procedures.

Consider the difference between “for every sample, there is an assay that works” “there is one assay that works for every sample.” Same words, different quantifie order, and the second is a much stronger claim than the first; which one is being determines what has to be searched for. In code, a variable’s identity depends on scope. In an enterprise workflow, whether an action is permitted may depend on approved which action for which account. Flatten these relationships into a bag features and you can erase precisely the distinction that decides the answer.

My expectation is that richer interfaces to such programs will often benefit from recurrence — from going around the loop more than once. An initial neural interpretation proposes some bindings; symbolic execution reveals an inconsiste the interpretation gets revised; a different program becomes relevant; and so on things settle. The small neural layers below and above the symbolic layer may ne take part in that back-and-forth rather than each making one irrevocable pass. T exactly the kind of iterative, local reconciliation of evidence that predictive codi built around, which is a large part of why I expect PC to come into its own here. are also established links between PC-style or equilibrium learning and ordinary gradient-based learning, under explicit conditions, so adopting PC needn’t mean abandoning what we know about backprop.[6]

That said, I don’t want to oversell PC (as much as I like it). We will need to try th experiments and see if PC sells itself in this context. BP can train recurrent computation too, including by differentiating through a finite sequence of infere steps, and PC doesn’t automatically make recurrence faster, more accurate or mo stable. My latest NESS retention and binding results in fact used BP, run throug

finite number of reliability-inference steps; they are not evidence of PC's superic
My earlier comparisons showed that PC was competitive in some settings — but
more like a respectable showing for PC rather than a universal win.[1,2] My guess
that PC will deserve a stronger role as the interpretation problem becomes more
tightly coupled, but it's a guess, and the decisive experiment has to separate two
questions that are easy to run together: whether recurrence helps at all, and whe
PC is the best way to learn it.

Plugging the sandwich into Hyperon, and giving it a history

Now let's think about the symbolic level a bit more. It can go back and forth with
Hyperon AtomSpace. If you have the whole sandwich wrapped in an Omega loop
no longer a frozen neural net interacting with a separate symbolic memory. A typ
program, a relation with provenance attached, or an uncertain hypothesis is som
the broader architecture already knows how to handle — it can live in an AtomS
be rewritten by MeTTa code, be reasoned about by PLN, be evolved by MOSES-s
program search — rather than being one more embedding whose meaning is pri
a single network. And the flow goes the other way too: the output of semantic rel
extraction can feed the symbolic layer now, and the output of Hyperon reasoning
feed it later, giving the neural predictor new computations to learn to exploit. To
plain about status, this two-way integration is what we *want* to build. The prototy
described above use supplied typed records and locally discovered programs, not
Hyperon connection.

Another next step I find especially appealing is a shared episodic memory attach
both the distilled bridge and the neural cap — the two slices of bread. Daniel
MacDonald's Indexed Attention Transformer proposal offers one concrete route:
retain raw experience together with selected historical, memory-conditioned neu
representations, and train how those records get read and written. The reason to
attach memory on both sides is that the two reads would do different jobs: an ear
read at the bridge helps interpret the situation before the symbolic layer gets to v

and a later read at the cap retrieves a relevant precedent or exception after the programs have exposed the situation's structure. Throughout, the raw evidence a provenance have to stay available, because an old interpretation is not automatic true belief. This is a proposal at present, not part of the measured results above.[

Over the last couple days one of my colleagues and his AI coding agent minions buil at systematic NESS research codebase around modular interfaces, including FabricPC backend for the predictive-coding side. The ambition is to be able to s substrates, symbolic mechanisms, memory systems, combiners and learning rule without rebuilding the whole experiment each time. The same modularity opens lateral experiments — a NESS cap above a time-series forecaster, say, or frozen language and vision models feeding a shared symbolic layer — which are going t quite fun to run.

What I want to learn next is fairly concrete. Do reusable programs still improve prediction when the neural baseline gets stronger? Can memory remain usable a interpretations that index it evolve? Can the upper components keep adapting w losing the ability to use what they learned before? And does recurrent predictive coding earn its computational cost?

Idealism, pragmatism, and the race

I think of NESS as an example of the flexibility of possible AGI designs, and also way we're mixing idealism with pragmatism in building hybrid architectures tha hopefully be made to outperform vanilla frontier LLMs fairly soon. ***We are in an A race here***. It's not necessarily the optimal dynamic to be operating in, but it's the There are optimal ways to do things, and replacing the whole transformer with a predictive-coding network designed from the ground up for neural-symbolic interfacing is probably a very good way. But we want to get to beneficial AGI qui and so it's worth exploring a variety of complementary routes to the neural side neural-symbolic AGI (as we also work to scale up and refine the symbolic Hyper side) … if you can get far enough by putting a neural-symbolic cap on existing op weights models, that may be what gets us there in time.

To reiterate the point about the race, my reasoning and feeling here is that: ***What need to do to get a beneficial future for our species is to have an open, decentralized AG an architecture capable of real moral agency, teach it human values in a sensible way, a it out on a globally distributed decentralized infrastructure as soon as possible*** — so tha open, decentralized, beneficial side of things is in the lead, ahead of the closed, proprietary, military, purely profit- and nationalism- driven side.

That's not to say the military won't use this stuff, or that we and others won't ma profit with it. It's to say that open and decentralized needs to be in the lead, and everyone else can play catch-up. And to get there, we can't necessarily wait arou totally replace backprop transformers with superior things. Unless someone coo a route even faster than the one I'm thinking of, which would be great.

We don't need to replace every synaptic weight in current open-weights models to build system that keeps learning continuously, and can serve as a close partner to symbolic AI neural-symbolic AGI architectures like Omega. We need the right division of labor between stable competence, revisable interpretation, explicit knowledge and persistent discovery. NESS is my most recent (and I think most interesting so far attempt to turn that question into an architecture — and then to flesh out and re the architecture via experimentation….

Explore the work

Research draft: See this folder for a series of (LLM-assisted) paper drafts going o the math and practicalities of NESS as currently conceived. Quick technical-ish overview here.

Code and contribution starting point: github.com/guilamacie/NESS

Sources and research notes

The numbered notes distinguish the completed prototype record from outside precedents and from proposed extensions.

1. NESS architecture. The NESS working-paper series, Paper 1: *Architecture and Working Principles* and Paper 2: *Mathematical Framework and Implemented Instantic* (September 2026). These internal research drafts define the five interfaces, typed readers, binding grammar, probabilistic consumers, and the distinct BP and PC implementations. They do not establish large-scale superiority.

2. Prototype results. NESS Paper 3: *Experimental Record and Evidence Audit*, espec Sections 11–12, together with the archived H3 v44/v45 results and, for the worke shared-witness example, the archived v43 program catalogue, test worlds and sav forecasts. All numerical values in this post come from that completed record; no experiment was rerun for the post. Recovery percentages measure reversal of a lc increase, not a fraction of all knowledge retained. Earlier experiments used diffe model families and data.

3. Alternatives to fixed deployment. P. Lewis et al., Retrieval-Augmented Gener for Knowledge-Intensive NLP Tasks (2020); A. Behrouz, P. Zhong and V. Mirrokr Titans: Learning to Memorize at Test Time (2025 preprint). These are precedents NESS experiments.

4. Efficient adaptation and distillation. E. J. Hu et al., LoRA: Low-Rank Adaptat Large Language Models (2021); G. Hinton, O. Vinyals and J. Dean, Distilling the Knowledge in a Neural Network (2015). They support the general building blocks a measured cost advantage for a large NESS system.

5. Structured scientific knowledge. Gene Ontology Consortium, Gene Ontology overview; S. Kim et al., PubChem 2025 update. These illustrate structured knowl resources; no NESS integration with either is claimed.

6. Predictive coding and equilibrium learning. B. Millidge, A. Tschantz and C. I Buckley, Predictive Coding Approximates Backprop along Arbitrary Computatic Graphs (2020); B. Scellier and Y. Bengio, Equilibrium Propagation: Bridging the (Between Energy-Based Models and Backpropagation (2017). Their learning rules assumptions are distinct; neither establishes a universal PC advantage.

7. Episodic read–write extension. D. MacDonald, *Indexed Attention Transformers: Continual In-Context Learning over an Expanding Experience Log*, revised internal cc manuscript, 24 September 2026. The dual attachment to the NESS bridge and cap proposed adaptation of that idea, not a completed experimental result.

8. Codebase and implementation scope. The NESS repository and its implementation-status record, consulted 27 September 2026. Repository capabili statements are developer-reported, not independently reproduced for this draft. ' new platform and its engineering demonstrations are distinct from the earlier H experiments summarized above.

Subscribe to Eurykosmotron

By Ben Goertzel · Launched 5 years ago

AGI, frontier science, maniacal metaphysics, decentralizationist politics, life and consciousness exten expansion, psi and psychedelics and etc. etc.

Type your email... Subscribe

By subscribing, you agree Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.



12 Likes · 2 Restacks

Discussion about this post

Comments Restacks



Write a comment...



Oleg Alexandrov 6d

"I read this as evidence that an explicit computation can earn its place inside a neural predict predictor's own terms. It is not evidence that symbolic features supply facts the network cou otherwise get at. And there's a result here I'd have preferred to go the other way: co-trainin alongside the programs did not increase the programs' incremental benefit."

This kind of experiments are very interesting and blur the line between fixed neural nets pro data and neural nets that modify themselves as they process the data.

First thing I must say is that folks who claim LLM are "frozen" while the human mind "continu rewires itself" likely greatly overstate their case. Rewiring is very expensive and very slow, an likely we do it only for most fundamental skills, while for the rest our brain tries to get by wi something shallower.

It still appears to me that keeping the neural net fixed, and doing all subsequent work in ext memory is more robust and better long term, even though Ben claims it works badly someti avoid catastrophic forgetting this way. You can do several passes over your data and over yo memories, and each time distilling or refining yet another insight, maybe with new data as w you add as yet another memory block.

Lastly, one should be a bit weary of trying "evolve" AI without continual supply of new data grounding, by simply optimizing its reasoning. We people, are very driven by specifics and do function well when going to the deep end. I suspect it would be the same for AI.

LIKE (1) REPLY



Harold Gielow 4d

Today, I felt like I was experiencing a little of Ben's Neural-Symbolic Sandwich construct. I ha was to me, and it seems to the area of art and invention, a novel idea; an idea for something invented. Well, I found it had been invented by my back and forth on the basic idea with the as I would alternate between one and the other, but not quite as I had imagined its instantia original idea evolved into a different new idea through my back and forth with these two AI First, I would not have made the leap from my original idea to the already invented and in w use machine without having had this conversation about my thought, which to me was nove I would not have then applied my original idea to the already extant invention to improve it particular use without this back and forth. In this role, I was the middleware, or mayo I suppo sandwich; that which blends separate things together, perhaps in novel ways, to solve a prol which problem set also evolved. This represents creativity, which often arises in humans fro inferences and hypotheses which come about from a a significant knowledge base of inform relationships subjected to what ifs and even flights of fanciful imagination. It might even be considered a playfulness in a search for new meaning about information relationships. I play role singularly, not noticing the AI assistants straying from their knowledge base to such cor but each tool played a different, unique and important role in this flight of investigative exp One AI tool, I will call it the less scientifically and fact based assistant, which I have often cau hallucinating, provided less pushback to possibilities, and more validation of the hypothetica checking its replies with the other AI, which I will call the "upper crust" of the sandwich, I go

hard facts; this is not possible for these scientific reasons, but my interactions with the lowe informed me of alternative possible paths of enquiry. What perhaps was lacking from this sa was a cool drink from the extended “matrix” of agents, models, and relationships to ask dow bites from both the upper and lower crusts. What it did allow me to see was, not the lack of the less scientifically informed AI, because when I fed it back the upper crust responses and on its math, it corrected through iterations, but a value in playing around in the realm of the before testing in the realm of the doable based on hard science, principles and math. In the came up with a novel improvement to a machine with a current real use and marketability. T marketability could, not saying it will, produce jobs in the future and improve people’s lives. the value propositions of human, AI real collaboration. They might also say something usefu non technical mind, about Ben’s Neural-Symbolic Sandwich construct.

LIKE REPLY

2 more comments...

© 2026 Ben Goertzel · Privacy · Terms · Collection notice
Substack is the home for great culture